ML_Lab8_UEN

October 12, 2026

1 Support Vector Machine (SVM)

1.1 Aim

Implement SVM classification and investigate kernels, feature scaling, and model evaluation.

1.2 Learning outcomes

By the end of this notebook, you should be able to: 1. Train and evaluate a linear SVM classifier. 2. Compare linear, polynomial, and RBF kernels.

1.3 1. Import the required libraries

We will use scikit-learn to train SVM models, NumPy to work with arrays, and Matplotlib to visualize the data and decision boundaries.

[ ]:
```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_blobs, make_moons, load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, ConfusionMatrixDisplay, classification_report
)

RANDOM_STATE = 42
```

1.4 2. Create a two-dimensional dataset

A two-dimensional dataset makes it possible to draw the decision boundary. We start with two fairly well-separated classes.

[ ]:
```
X, y = make_blobs(
    n_samples=300,
    centers=2,
    n_features=2,
```

```
    cluster_std=1.4,
    random_state=RANDOM_STATE
)
```

[ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=RANDOM_STATE, stratify=y
)

print(X_train.shape)
```

[ ]:
```
print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
print("Number of features:", X_train.shape[1])



plt.figure(figsize=(7, 5))
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap="coolwarm",
            edgecolors="k", alpha=1)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Training data")
plt.show()
```

1.5 3. Train a linear SVM

We will first standardize the features. The scaler is fitted using only the training data, then applied to the test data. This avoids using information from the test set during preprocessing.

[ ]:
```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

linear_svm = SVC(kernel="linear", C=1)
linear_svm.fit(X_train_scaled, y_train)

y_pred = linear_svm.predict(X_test_scaled)

print("Accuracy :", round(accuracy_score(y_test, y_pred), 4))
print("Precision:", round(precision_score(y_test, y_pred, zero_division=0), 4))
print("Recall   :", round(recall_score(y_test, y_pred, zero_division=0), 4))
print("F1-score :", round(f1_score(y_test, y_pred, zero_division=0), 4))
print("\nConfusion matrix:")
print(confusion_matrix(y_test, y_pred))
```

1.6 4. Compare kernels on nonlinear data

The `make_moons()` dataset has two interleaving crescent-shaped classes. It is useful for demonstrating why a straight decision boundary may be inadequate.

[ ]:
```
X_moons, y_moons = make_moons(
    n_samples=500, noise=0.25, random_state=RANDOM_STATE
)

X_m_train, X_m_test, y_m_train, y_m_test = train_test_split(
    X_moons, y_moons, test_size=0.25, random_state=RANDOM_STATE,
    stratify=y_moons
)

moon_scaler = StandardScaler()
X_m_train_scaled = moon_scaler.fit_transform(X_m_train)
X_m_test_scaled = moon_scaler.transform(X_m_test)

plt.figure(figsize=(7, 5))
plt.scatter(X_m_train_scaled[:, 0], X_m_train_scaled[:, 1],
            c=y_m_train, cmap="coolwarm", edgecolors="k", s=30)
plt.xlabel("Scaled feature 1")
plt.ylabel("Scaled feature 2")
plt.title("Training data: make_moons")
plt.show()
```

[ ]:
```
import numpy as np
import matplotlib.pyplot as plt

def plot_svm_boundary(model, X_plot, y_plot, title):
    x_min, x_max = X_plot[:, 0].min() - 0.8, X_plot[:, 0].max() + 0.8
    y_min, y_max = X_plot[:, 1].min() - 0.8, X_plot[:, 1].max() + 0.8

    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 300),
        np.linspace(y_min, y_max, 300)
    )
    grid = np.c_[xx.ravel(), yy.ravel()]
    decision_values = model.decision_function(grid).reshape(xx.shape)

    plt.figure(figsize=(7, 5))
    plt.contourf(xx, yy, decision_values, levels=30, cmap="coolwarm", alpha=0.
 ↪22)
    plt.contour(xx, yy, decision_values, levels=[-1, 0, 1],
                colors="black", linestyles=["--", "-", "--"])
    plt.scatter(X_plot[:, 0], X_plot[:, 1], c=y_plot, cmap="coolwarm",
                edgecolors="k", s=35)
    plt.scatter(model.support_vectors_[:, 0], model.support_vectors_[:, 1],
```

```
                s=150, facecolors="none", edgecolors="gold", linewidths=2,
                label="Support vectors")
    plt.xlabel("Scaled feature 1")
    plt.ylabel("Scaled feature 2")
    plt.title(title)
    plt.legend()
    plt.show()
```

Train linear, polynomial, and RBF SVM models. All models use the same training and test samples.

[ ]:
```
kernel_models = {
    "Linear": SVC(kernel="linear", C=100),
    "Polynomial (degree 2)": SVC(kernel="poly", degree=2, C=100),
    "Polynomial (degree 3)": SVC(kernel="poly", degree=3, C=100),
    "RBF": SVC(kernel="rbf", C=100, gamma="scale")
}

kernel_results = []

for name, model in kernel_models.items():
    model.fit(X_m_train_scaled, y_m_train)
    pred = model.predict(X_m_test_scaled)
    kernel_results.append((
        name,
        accuracy_score(y_m_test, pred),
        precision_score(y_m_test, pred, zero_division=0),
        recall_score(y_m_test, pred, zero_division=0),
        f1_score(y_m_test, pred, zero_division=0),
        len(model.support_vectors_)
    ))
    plot_svm_boundary(model, X_m_train_scaled, y_m_train, name + " kernel")

print(f"{'Kernel':<24} {'Accuracy':>9} {'Precision':>10} {'Recall':>8} {'F1':
 ↪>8} {'SVs':>6}")
for row in kernel_results:
    print(f"{row[0]:<24} {row[1]:>9.3f} {row[2]:>10.3f} {row[3]:>8.3f} {row[4]:
 ↪>8.3f} {row[5]:>6}")
```

1.7 5. Why feature scaling matters

SVM depends on distances and dot products, so features with much larger numerical ranges can dominate the model. This section compares a model trained on features with different scales before and after standardization.

We create a two-feature dataset and deliberately multiply the second feature by a large constant.

[ ]:
```
X_scale, y_scale = make_blobs(
    n_samples=400, centers=2, n_features=2, cluster_std=2.0,
```

```
    random_state=RANDOM_STATE
)
X_scale[:, 1] = X_scale[:, 1] * 1000

X_s_train, X_s_test, y_s_train, y_s_test = train_test_split(
    X_scale, y_scale, test_size=0.25, random_state=RANDOM_STATE,
    stratify=y_scale
)

# Experiment A: no scaling
svm_unscaled = SVC(kernel="rbf", C=1.0, gamma="scale")
svm_unscaled.fit(X_s_train, y_s_train)
unscaled_acc = svm_unscaled.score(X_s_test, y_s_test)

# Experiment B: standardization
scale_scaler = StandardScaler()
X_s_train_scaled = scale_scaler.fit_transform(X_s_train)
X_s_test_scaled = scale_scaler.transform(X_s_test)

svm_scaled = SVC(kernel="rbf", C=1.0, gamma="scale")
svm_scaled.fit(X_s_train_scaled, y_s_train)
scaled_acc = svm_scaled.score(X_s_test_scaled, y_s_test)

print("Test accuracy without scaling:", round(unscaled_acc, 4))
print("Test accuracy with scaling   :", round(scaled_acc, 4))
```

1.8 6. Final evaluation on a real dataset

Now use the Breast Cancer Wisconsin dataset for a more realistic classification task. We use a separate training and test set, standardize the features, and evaluate a linear SVM.

[ ]:
```
cancer = load_breast_cancer()
X_cancer = cancer.data
y_cancer = cancer.target

X_c_train, X_c_test, y_c_train, y_c_test = train_test_split(
    X_cancer, y_cancer, test_size=0.20, random_state=RANDOM_STATE,
    stratify=y_cancer
)

cancer_scaler = StandardScaler()
X_c_train_scaled = cancer_scaler.fit_transform(X_c_train)
X_c_test_scaled = cancer_scaler.transform(X_c_test)

cancer_svm = SVC(kernel="linear", C=1.0)
cancer_svm.fit(X_c_train_scaled, y_c_train)
cancer_pred = cancer_svm.predict(X_c_test_scaled)
```

```
print(classification_report(
    y_c_test, cancer_pred, target_names=cancer.target_names, zero_division=0
))

ConfusionMatrixDisplay.from_predictions(
    y_c_test, cancer_pred, display_labels=cancer.target_names,
    cmap="Blues"
)
plt.title("Confusion matrix: Breast Cancer dataset")
plt.show()
```

1.8.1 Task 1

1. Which class has the higher recall?
2. What does a false negative mean in this dataset?

1.8.2 Task 2: Investigating the Effect of Kernels on the Breast Cancer Dataset

In the previous experiment, you used a linear SVM to classify the Breast Cancer dataset. Now, investigate how different kernel functions affect classification performance.

Instructions:

Train four SVM classifiers using the following kernels:

Linear

Polynomial (degree 2)

Polynomial (degree 3)

RBF

Use the same training and test sets created in Section 6, along with the standardized features.

For each model, record:

Training accuracy

Test accuracy

Total number of support vectors

Present your results in a comparison table.

Questions: Which kernel achieves the highest test accuracy?

Which kernel uses the largest number of support vectors?

Does a higher number of support vectors necessarily indicate better classification performance?

1.8.3 Task 3: Investigating the Impact of C on the Number of Support Vectors

The parameter (C) controls the trade-off between maximizing the margin and penalizing margin violations. Investigate how changing (C) affects the number of support vectors.

Instructions:

Use the linear kernel and experiment with the following values:

C = {0.01, 0.1, 1, 10, 100}

Train a separate SVM for each value of (C) using all kernels, using the same standardized training data.

Record the training accuracy, test accuracy, and total number of support vectors for each model.

Plot a graph showing (C) on the x-axis and the total number of support vectors on the y-axis. Use a logarithmic scale for the x-axis.

Questions:

How does the number of support vectors change as (C) increases?

Which value of (C) gives the highest test accuracy?

Does increasing (C) always reduce the number of support vectors? Explain your observations.

1.8.4 Task 4: Comparing Kernel and C Effects

Based on the results obtained in Tasks 2 and 3, answer the following questions:

Which kernel would you recommend for this dataset, based on test accuracy and model complexity?

Does changing (C) affect the number of support vectors in the same way for every dataset or kernel?

[ ]: